

Action Penalties Decide Tool Use in Cost-Aware Agentic RAG

Parsuram Pradhan

parsuram.pradhan@yobytech.com

Abstract

Agentic retrieval can search again, rewrite the query, reorder passages, or check a draft before it answers. Each step has a price, but most learned search agents optimize answer quality under a step cap. We train a controller on five typed actions—retrieve, rewrite, rerank, verify, and stop—with a reward that subtracts measured token dollars, retrieval dollars, latency, and a flat charge per action from answer quality, grounding, and calibration. On 150 HotpotQA and 150 Natural Questions items, with BM25 over 80,000 passages and Qwen2.5-3B-Instruct, using every tool raises exact match from 100/300 to 105/300. That quality gain is worth about 0.03 reward. The flat action charge for the same trajectory is 0.10, and the measured dollar and latency gap is about 0.002. A 261-parameter REINFORCE policy therefore matches one-shot retrieval on all 300 test questions. The same trainer spends its eight-step budget when every penalty is removed, so the collapse is the objective. Sweeping the dollar weight changes the last-epoch action string, yet every learned policy emits one fixed string on every question, and the checkpoint with the best training reward is again one-shot retrieval. An oracle that keeps the better of those two answers scores 124/300. The headroom is real. The policies we trained do not use it.

1 Introduction

A retrieval-augmented generator answers from passages it has fetched (Lewis et al. 2020). An agentic loop can do more than fetch once. It can rewrite the query, retrieve again, reorder the pool, test whether the draft is supported, or stop. In a serving stack those operations do not share a price. A redundant search and a cheap reorder can look identical to a controller that sees only exact match.

Recent reinforcement-learning agents learn when to search, and they learn it for answer quality (Jin et al. 2025; Gandhi et al. 2026). Turn count appears as an efficiency bonus. Dollars and latency do not. Open-loop routers choose a depth before they see the passages (Jeong et al. 2024). Hand-written critics decide with thresholds (Asai et al. 2024; Yan et al. 2024). None of these tells a closed-loop policy when a priced tool is worth its bill.

We study that decision in a small, fully instrumented environment. The action set is frozen at retrieve, rewrite,

rerank, verify, and stop. The reward is explicit: answer quality, grounding, and calibration, minus a FinOps price card, a hallucination penalty, and a per-action charge. The policy is a 261-parameter multilayer perceptron trained with REINFORCE on 100 questions and scored, frozen, on a locked set of 300. The generator, retriever, and verifier stay fixed, so a change in exact match is a change in control.

The measurements support three claims.

- Under the default weights, extra tools are negative expected value. A full tool trajectory gains about 0.03 quality and pays 0.10 in action charges. One-shot retrieval has the highest reward.
- The learner follows that arithmetic. With the action charge left on, argmax evaluation is retrieve-then-stop on 300/300 questions, including when dollar and latency weights are zero. With every penalty removed, the same network spends the step budget.
- Dollar weight changes the last-epoch string, and that string is constant across questions. The checkpoint selected by greedy training reward is one-shot retrieval at every dollar weight we tried. A per-question choice between one-shot retrieval and the zero-dollar recipe would answer 124/300, which no trained policy reaches.

2 Related Work

Retrieval-augmented generation conditions a generator on retrieved text (Lewis et al. 2020). Agentic controllers interleave that retrieval with other tools. ReAct alternates reasoning traces and actions (Yao et al. 2023). Self-RAG learns tokens for when to retrieve and when a citation supports a sentence (Asai et al. 2024). CRAG uses a retrieval evaluator to trigger a second search or a web fallback (Yan et al. 2024). Adaptive-RAG assigns each query to no retrieval, single-step retrieval, or multi-step retrieval before any passage is observed, which makes it an open-loop contrast to a controller that can react to a bad first search (Jeong et al. 2024).

Search-R1 trains the language model itself, with an outcome reward, to issue searches and reason over the results (Jin et al. 2025). GRASP is the closest typed-action neighbor: a GRPO policy over semantic search, keyword search, and paragraph expansion, rewarded for answer F1, grounded reading, complementary search, and turn efficiency (Gandhi

et al. 2026). GRASP does not price rewrite, rerank, or verification, and its efficiency term counts turns. We keep a much smaller policy and ask a prior question: once rewrite, rerank, and verify have measured dollar costs, does a learned controller spend them only when they pay?

We do not claim a comparison against a reimplementaion of Search-R1, GRASP, or Adaptive-RAG on this stack. Those systems are the reason to price tools. The experiments below isolate the price.

3 Environment and Reward

An episode is one question. At each step the controller picks one action. The episode ends at stop, or when the step or budget cap forces a stop. The five actions are the whole interface.

- **Retrieve** runs BM25 (Robertson and Zaragoza 2009) and appends the top five passages.
- **Rewrite** asks the generator for a new query string.
- **Rerank** reorders the current pool by lexical overlap with the query.
- **Verify** runs a lexical natural-language-inference check of the current draft against the passages and returns support, neutral, or contradiction.
- **Stop** emits a short answer, or abstains. Yes/no questions must be answered yes or no.

Caps are eight steps, three retrieves, two rewrites, two reranks, and two verifies. The price card meters local Qwen2.5-3B-Instruct tokens at \$0.15 and \$0.60 per million input and output tokens, BM25 at 5×10^{-5} dollars per call, and latency at 10^{-3} dollars per second scaled by a latency weight (Qwen Team 2024). Absolute dollars are an instrumentation scale for a local model. Ratios are what the policy can learn.

The observation for the reported policies is ten numbers: tanh-scaled mean retrieval score, evidence count, remaining steps, remaining dollar budget, verifier support, verifier contradiction, and the four tool counts. There is no dataset indicator and no passage text in the control head. The generator still reads the passages. Illegal actions are masked: with no evidence the policy cannot rerank, verify, or stop.

The terminal reward is

$$R = \alpha Q_{\text{ans}} + \beta Q_{\text{ground}} + \gamma Q_{\text{cal}} - \lambda(C_{\text{tok}} + C_{\text{ret}}) - C_{\text{lat}} - P_{\text{hall}} - P_{\text{act}} - P_{\text{bud}} + B. \quad (1)$$

Q_{ans} is the average of exact match and token F1. Q_{ground} mixes claim–evidence overlap with, on HotpotQA, recall of gold supporting titles. C_{tok} and C_{ret} are price-card dollars. C_{lat} already includes the latency weight. P_{hall} penalizes unsupported and contradicted claims, with an extra weight if verify was available and unused. P_{bud} is a unit penalty if a hard budget breaks. The unused-budget bonus B is granted only when $Q_{\text{ans}} \geq 0.5$, so a cheap wrong answer does not collect an efficiency credit. The action charge is $P_{\text{act}} = \rho \max(0, n - 1)$ for n actions in the episode.

Default weights are $\alpha=1$, $\beta=0.4$, $\gamma=0.15$, $\lambda=2$, latency weight 0.5, hallucination weight 0.5, and $\rho=0.02$. Calibration is not a second copy of exact match. A correct answer scores +0.3. Abstaining scores +0.6 only when the passage list is empty, the mean retrieval score is below 3, or verify returns contradiction; any other abstention scores −0.2. A wrong answer scores −0.4 with evidence and −0.2 without it. Abstaining on a string that already matches the gold answer scores −0.5. A policy cannot farm reward by refusing whenever it would have been wrong.

4 Learning

The policy is a multilayer perceptron, $10 \rightarrow 16 \rightarrow 5$, with 261 parameters. Width above 32 is rejected: the training slice is 100 questions. We optimize with REINFORCE (Williams 1992). The return is the terminal reward in Equation 1. The baseline is an exponential moving average of that return. The entropy bonus coefficient is 0.01, the optimizer is Adam, and the learning rate is 0.003. Training reads only the 100-example training file. The evaluation file is refused by name.

At test time the action is the mode of the masked softmax. Sampling during training can visit tools that the mode never selects, so we also roll out the mode on the training questions after each epoch of the dollar sweep. The checkpoint we treat as the result is the epoch with the highest greedy training reward. It is chosen before the 300-question evaluation is opened. The last epoch is reported beside it, and it is not the selected policy when the two differ.

5 Experimental Setup

The evaluation set is locked at 300 questions: 150 HotpotQA distractor items (Yang et al. 2018) and 150 Natural Questions items (Kwiatkowski et al. 2019). The training slice is 60 HotpotQA and 40 Natural Questions drawn from the same sources and disjoint from the evaluation ids. The shared index holds 80,000 passages: HotpotQA contexts, unused HotpotQA distractors, and the DPR 100-word Wikipedia passages released with Natural Questions (Karpukhin et al. 2020; Gao et al. 2022). Gold text is a Wikipedia passage. It is not a sentence of the form “the answer is.” BM25 recall at five is 0.927 on HotpotQA (11 misses) and 0.587 on Natural Questions (62 misses), so a second retrieval can matter.

The generator is Qwen2.5-3B-Instruct at temperature 0 (Qwen Team 2024). The verifier is lexical, not a cross-encoder and not the generator judging itself. Three frozen controllers define the corners. One-shot retrieval fetches once and stops. The rule policy reranks and verifies when the mean BM25 score is at least 3, and it stops after verify on every question in this run. The max-tool policy spends three retrieves, one rewrite, one rerank, and one verify. Reward-weight ablations that keep this rule policy fixed are a scoring check: exact match does not move, and only the scalar does. Every learned number below is a frozen argmax pass over the same 300 questions. We report one seed (42).

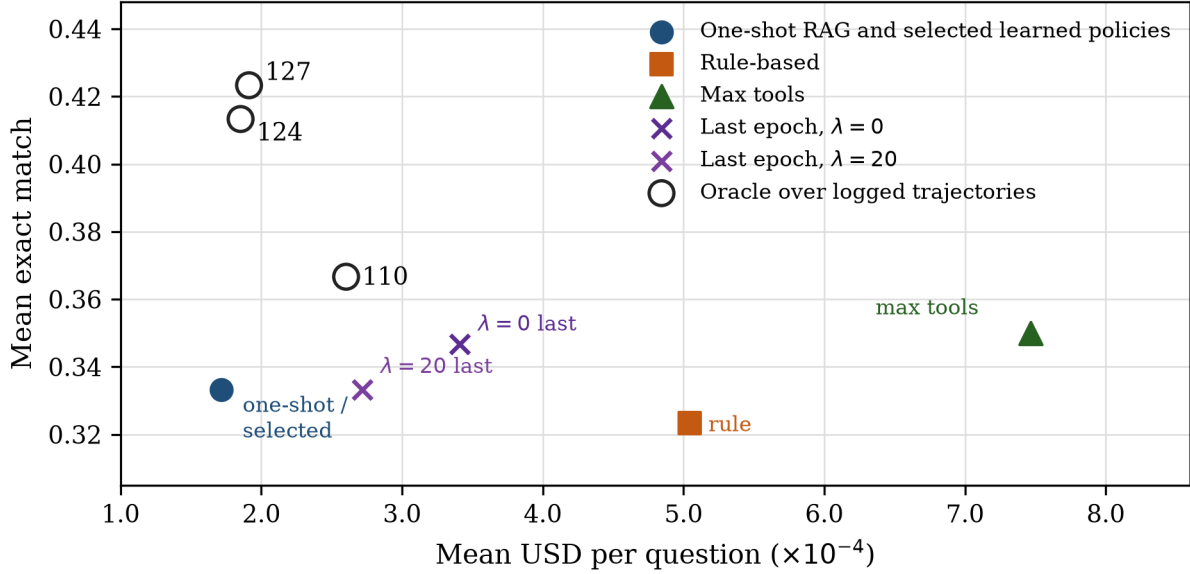


Figure 1: Exact match against mean dollars per question. Squares are frozen controllers. The filled circle is every selected learned checkpoint; all three sit on one-shot retrieval. Crosses are last-epoch recipes that the training reward did not select. Hollow circles are oracles over trajectories we already logged, not learned policies: 110 uses one-shot retrieval on HotpotQA and the zero-dollar recipe on Natural Questions; 124 keeps the better of those two answers on each question; 127 may also use the max-tool trajectory.

6 Results

Extra Tools Buy Five Answers

Table 1 splits the frozen controllers by dataset. Overall exact match is a mix of a multi-hop slice near 0.39 and a single-hop slice near 0.28, so we do not rank policies by the pooled number alone.

The max-tool policy recovers a net of two HotpotQA answers and three Natural Questions answers over one-shot retrieval, and its overall token F1 rises from 0.397 to 0.422. Mean cost rises by a factor of about 4.3, from 1.72×10^{-4} to 7.47×10^{-4} dollars, and mean latency from about 1.0 s to 3.4 s. Reward falls from 0.580 to 0.509.

The components say why. From one-shot retrieval to the max-tool policy, Q_{ans} rises by 0.021 (0.365 to 0.386), Q_{ground} by 0.018 (0.647 to 0.665), and Q_{cal} by 0.009 (-0.137 to -0.128). With the default weights that is a quality gain of 0.030. The action charge rises from 0.02 to 0.12. Scaled dollars and latency together rise by about 0.002. A quality term of 0.03 against a charge of 0.10 is a reward loss of about 0.07, which matches the observed gap. Of the penalty that separates the two trajectories, about 99 percent is ρ and about 1 percent is the price card. One extra retrieval costs roughly 3×10^{-4} in the $\lambda=2$ dollar term and 0.02 in ρ , so the flat charge is about sixty times the measured dollar price of that call. The rule policy, which always pays for a rerank and a verify, lands between them on both cost and reward and does not improve exact match.

On a stratified 100-question rescore of the same rule trajectories, exact match stays 0.34. Answer-only reward is 0.362,

adding grounding raises it to 0.547, and the full default objective scores 0.499. The weights change the scalar. They do not change a frozen controller.

The Learner Follows the Charge

Table 2 is the learned policy under objectives that turn pieces of Equation 1 off. The network is the same size in every row.

Under the default objective the training samples still use tools: over five epochs, mean training steps stay near 4 and mean verify calls stay between 0.39 and 0.55. The mode does not. Argmax evaluation is retrieve-then-stop on 300/300 questions, with the same predictions as one-shot retrieval. Entropy during training hid a retrieve-then-stop mode.

Zeroing dollar and latency weights, while leaving $\rho=0.02$, does not move that mode. The training curve still samples tools. The evaluation is again retrieve-then-stop, with identical predictions. The price card is not what pins the policy.

Removing every penalty does move it. Trained only on Q_{ans} , the mode uses all eight steps on 300/300 questions: three retrieves, two rewrites, and two verifies. Exact match rises from 100/300 to 102/300 (HotpotQA 60/150, Natural Questions 42/150), with 9 recoveries and 7 regressions. The trainer can leave two steps. It does so when extra steps cannot lower the return.

Figure 1 and the last five rows of Table 2 place the dollar weight on either side of the quality gap. At $\lambda=80$, the extra max-tool dollars are about $80 \times 5.8 \times 10^{-4} \approx 0.046$, which is larger than the 0.03 quality those tools buy, and ρ is also on. Greedy training is retrieve-then-stop on all 40 epochs, and so is the evaluation. At $\lambda=20$ and $\rho=0$, the same extra

Policy	Hotpot	NQ	EM	Correct	\$	Steps	Reward
One-shot	59/150	41/150	0.333	100/300	1.72×10^{-4}	2	0.580
Rule	56/150	41/150	0.323	97/300	5.04×10^{-4}	4	0.531
Max tools	61/150	44/150	0.350	105/300	7.47×10^{-4}	7	0.509

Table 1: Frozen controllers on the 300-question evaluation. Hotpot is HotpotQA exact-match count. NQ is Natural Questions. EM and reward are means. One-shot retrieval has the highest reward. The max-tool policy has the highest exact match.

Objective	Checkpoint	EM	Correct	Action string
Default, $\rho=0.02$, $\lambda=2$	5 epochs	0.333	100/300	retrieve, stop
$\lambda=\mu=0$, $\rho=0.02$	5 epochs	0.333	100/300	retrieve, stop
Answer only	5 epochs	0.340	102/300	cap: 3 retrieve, 2 rewrite, 2 verify
$\lambda=0$, $\rho=0$	best, epoch 25	0.333	100/300	retrieve, stop
$\lambda=0$, $\rho=0$	last, epoch 40	0.347	104/300	1 retrieve, 2 rewrite, 2 verify
$\lambda=20$, $\rho=0$	best, epoch 15	0.333	100/300	retrieve, stop
$\lambda=20$, $\rho=0$	last, epoch 40	0.333	100/300	retrieve, retrieve, retrieve, stop
$\lambda=80$, $\rho=0.02$	best = last	0.333	100/300	retrieve, stop

Table 2: Frozen argmax evaluation of the learned policy. Each row is one action string on all 300 questions. The selected checkpoint is the best greedy reward on the 100 training questions. Answer only removes every penalty, including ρ . The $\lambda=20$ last epoch repeats the same BM25 call three times.

dollars cost about 0.012 and are still smaller than 0.03, so untargeted tools are positive expected value on the frozen gap. The selected checkpoint, epoch 15, is retrieve-then-stop. The last epoch retrieves three times and stops. The three calls return the same passage ids, the answers match one-shot retrieval on 300/300 questions, and the spend rises from 1.72×10^{-4} to 2.72×10^{-4} dollars. At $\lambda=0$ and $\rho=0$, tools are free. The selected checkpoint is again epoch 25, retrieve-then-stop, training exact match 0.43. The last epoch is a six-step recipe—one retrieve, two rewrites, two verifies—with training exact match 0.40 and evaluation exact match 0.347 (104/300). On the training questions that recipe is three answers worse. On the evaluation set it is four answers better, and the split is uneven: HotpotQA falls from 59/150 to 53/150 while Natural Questions rises from 41/150 to 51/150. We do not treat 0.347 as the selected result. The selection rule, fixed before looking at the 300, returns one-shot retrieval.

Constant Recipes Leave a Per-Question Gap

Every learned evaluation in Table 2 uses one action sequence on all 300 questions. The policy does not branch on the retrieval score, the passage count, or the verifier label.

The verifier is not vacuous. On the rule policy’s trajectories it returns 14 contradictions, 31 neutrals, and 105 supports on HotpotQA, and 0 contradictions, 18 neutrals, and 132 supports on Natural Questions. The next action after verify is stop on 300/300 questions. The zero-dollar last epoch does call verify. Its final labels are 14 contradictions, 55 neutrals, and 231 supports. After a non-supporting label the next action is a second verify or stop, never a new retrieval or rewrite. Flipping the contradiction bit on a real post-verify observation changes action probabilities by at most 0.0068, and the mode stays verify either way.

Against one-shot retrieval, that six-step recipe repairs 6

HotpotQA answers and breaks 12, and repairs 18 Natural Questions answers and breaks 8. The net is +4. An oracle that keeps the better answer on each question scores 124/300 at a mean cost of 1.85×10^{-4} dollars, close to one-shot retrieval, because it pays for the long trajectory only where the answer changes. Restricting the oracle to one-shot retrieval on HotpotQA and the six-step recipe on Natural Questions scores 110/300 at 2.60×10^{-4} dollars, about a third of the max-tool bill. Allowing the max-tool trajectory as a third option scores 127/300. Single policies in this study span 100 to 105 correct. The 24-answer gap from 100 to 124 is the room a controller has if it reads the question in front of it. A dataset bit would be enough for the jump to 110, and the observation does not contain one. The jump to 124 requires a per-question choice. Nothing we trained makes that choice.

7 Limitations

The policy has 261 parameters and never reads passage text, so this is a study of the reward and of credit assignment in a compact controller, not of GRPO fine-tuning of the generator. We report one seed and 100 training questions. The verifier and the reranker are lexical. The price card’s absolute level is a local-model scale; the finding is the ratio of ρ to that scale. A neural verifier, a dense retriever, or a generator-sized policy could change the 0.03 quality gap, which would move the break-even dollar weight. We did not run that comparison. Adaptive-RAG and GRASP remain external reference points, not rows in Table 1.

8 Conclusion

Priced tool use in this environment is decided by the flat action charge. Under the default weights a full tool trajectory buys about five exact-match answers and pays about

0.10 in action charges, against a measured dollar and latency gap of about 0.002. REINFORCE on a 261-parameter policy matches one-shot retrieval whenever that charge, or a large dollar weight, makes extra tools a losing bet, and it spends the step budget when the bet cannot lose. Sweeping the dollar weight changes the last epoch’s action string and does not produce a policy that branches on evidence. The selected checkpoints are one-shot retrieval. An oracle over the trajectories already in hand answers 124 of 300 questions. Closing any of that gap requires a controller that changes its action when the retrieval or the verifier changes, trained against an objective in which a useful tool call is cheaper than the answer it buys.

Ethical Statement

A cost term can be satisfied by refusing to answer. The calibration scores in Section 3 penalize abstention when passages are already good enough, and they penalize a confident wrong answer more than an answer with no evidence. The learned policies in this paper do not exploit abstention: the selected checkpoints match one-shot retrieval, including its errors. They also ignore contradiction labels. Deploying that behavior would ship unsupported answers at the price of a single search. We report the failure and do not present these checkpoints as a serving policy.

References

- Asai, A.; Wu, Z.; Wang, Y.; Sil, A.; and Hajishirzi, H. 2024. Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. In *International Conference on Learning Representations*.
- Gandhi, V.; Lee, J.; Todmal, S.; Dernoncourt, F.; Rossi, R. A.; Wang, Z.; and Lan, A. 2026. GRASP: Granularity-Aware Search Policy for Agentic RAG. arXiv:2607.10463.
- Gao, L.; Ma, X.; Lin, J.; and Callan, J. 2022. Tevatron: An Efficient and Flexible Toolkit for Dense Retrieval. arXiv:2203.05765.
- Jeong, S.; Baek, J.; Cho, S.; Hwang, S. J.; and Park, J. C. 2024. Adaptive-RAG: Learning to Adapt Retrieval-Augmented Large Language Models through Question Complexity. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 7036–7050.
- Jin, B.; Zeng, H.; Yue, Z.; Yoon, J.; Arik, S.; Wang, D.; Zamani, H.; and Han, J. 2025. Search-R1: Training LLMs to Reason and Leverage Search Engines with Reinforcement Learning. arXiv:2503.09516.
- Karpukhin, V.; Oğuz, B.; Min, S.; Lewis, P.; Wu, L.; Edunov, S.; Chen, D.; and Yih, W.-t. 2020. Dense Passage Retrieval for Open-Domain Question Answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, 6769–6781.
- Kwiatkowski, T.; Palomaki, J.; Redfield, O.; Collins, M.; Parikh, A.; Alberti, C.; Epstein, D.; Polosukhin, I.; Devlin, J.; Lee, K.; Toutanova, K.; Jones, L.; Kelcey, M.; Chang, M.-W.; Dai, A. M.; Uszkoreit, J.; Le, Q.; and Petrov, S. 2019. Natural Questions: A Benchmark for Question Answering Research. *Transactions of the Association for Computational Linguistics*, 7: 452–466.
- Lewis, P.; Perez, E.; Piktus, A.; Petroni, F.; Karpukhin, V.; Goyal, N.; Küttler, H.; Lewis, M.; Yih, W.-t.; Rocktäschel, T.; Riedel, S.; and Kiela, D. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems*, volume 33, 9459–9474.
- Qwen Team. 2024. Qwen2.5 Technical Report. arXiv:2412.15115.
- Robertson, S.; and Zaragoza, H. 2009. The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval*, 3(4): 333–389.
- Williams, R. J. 1992. Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning. *Machine Learning*, 8(3–4): 229–256.
- Yan, S.-Q.; Gu, J.-C.; Zhu, Y.; and Ling, Z.-H. 2024. Corrective Retrieval Augmented Generation. arXiv:2401.15884.
- Yang, Z.; Qi, P.; Zhang, S.; Bengio, Y.; Cohen, W. W.; Salakhutdinov, R.; and Manning, C. D. 2018. HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, 2369–2380.
- Yao, S.; Zhao, J.; Yu, D.; Du, N.; Shafran, I.; Narasimhan, K.; and Cao, Y. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations*.